
```
>>> import decimal
>>> decimal.getcontext().prec
28
>>> decimal.getcontext().prec = 2
>>> decimal.Decimal(1) / 3
Decimal('0.33')
```

The `decimal` module provides you with fine control over how numbers interact with each other. The `decimal` module is documented in full at <https://docs.python.org/3/library/decimal.html>.

Don't Chain Inequality `!=` Operators

Chaining comparison operators like `18 < age < 35` or chaining assignment operators like `six = halfDozen = 6` are handy shortcuts for `(18 < age) and (age < 35)` and `six = 6; halfDozen = 6`, respectively.

But don't chain the `!=` comparison operator. You might think the following code checks whether all three variables have different values from each other, because the following expression evaluates to `True`:

```
>>> a = 'cat'
>>> b = 'dog'
>>> c = 'moose'
>>> a != b != c
True
```

But this chain is actually equivalent to `(a != b) and (b != c)`. This means that `a` could still be the same as `c` and the `a != b != c` expression would still be `True`:

```
>>> a = 'cat'
>>> b = 'dog'
>>> c = 'cat'
>>> a != b != c
True
```

This bug is subtle and the code is misleading, so it's best to avoid using chained `!=` operators altogether.